

Question 6

1 Introduction

The objective of this project was to design and implement a fuzzy logic controller for a simple unicycle robot. The robot operates on a 2D plane and is tasked with navigating from an initial state (x_0, y_0, θ_0) to a desired goal state (x_g, y_g, θ_g) . The robot's movement is governed by two control inputs: linear velocity (v) and angular velocity (ω).

- **Robot Kinematic Model:**

$$\begin{aligned}\dot{x} &= v \cos(\theta) \\ \dot{y} &= v \sin(\theta) \\ \dot{\theta} &= \omega\end{aligned}$$

- **Initial State:**

$$(x_0, y_0, \theta_0) = (0, 0, 0)$$

- **Goal State:**

$$(x_g, y_g, \theta_g) = \left(2, 2, \frac{\pi}{2}\right)$$

- **Control Constraints:**

$$\begin{aligned}0 &\leq v \leq 0.5 \quad (\text{m/s}) \\ -1 &\leq \omega \leq 1 \quad (\text{rad/s})\end{aligned}$$

The fuzzy controller was designed to determine the appropriate v and ω based on two system inputs: the robot's **distance** to the goal and the **angle error** relative to the target.

2 Fuzzy Logic Controller Design

The controller was developed using the `scikit-fuzzy` library in Python and is structured around three main components: Fuzzification, Inference, and Defuzzification.

2.1 Fuzzification (Defining Fuzzy Sets)

Fuzzification is the process of converting crisp numerical inputs into fuzzy linguistic variables. The two inputs and two outputs of our system were defined as follows:

- **Inputs (Antecedents):**

- **Distance to Goal (d):** This is the Euclidean distance from the robot's current position (x, y) to the goal (x_g, y_g) . It was categorized into three fuzzy sets: **'close'**, **'moderate'**, and **'far'**.
- **Angle Error (α):** This is the difference between the desired heading (the angle to the goal) and the robot's current orientation (θ). It was categorized into five fuzzy sets: **'negative_large'**, **'negative_small'**, **'zero'**, **'positive_small'**, and **'positive_large'**.

- **Outputs (Consequents):**

- **Linear Velocity (v):** This output controls the robot’s forward speed. It was categorized into three fuzzy sets: **'slow'**, **'medium'**, and **'fast'**.
- **Angular Velocity (ω):** This output controls the robot’s turning speed. It was categorized into five fuzzy sets: **'negative_fast'**, **'negative_slow'**, **'zero'**, **'positive_slow'**, and **'positive_fast'**.

The membership functions for these sets were defined using trapezoidal and triangular shapes to provide smooth transitions between linguistic categories.

2.2 Fuzzy Rules (Inference Engine)

The **inference engine** uses a set of linguistic rules to map the input fuzzy sets to the output fuzzy sets. The core logic of the controller is captured in these rules, which were designed to guide the robot intuitively. For example:

- **If distance is 'far' and angle_error is 'zero', then linear_velocity is 'fast' and angular_velocity is 'zero'.** (Go straight and fast when far from the goal and facing the right direction.)
- **If distance is 'moderate' and angle_error is 'positive_large', then linear_velocity is 'slow' and angular_velocity is 'positive_fast'.** (Slow down and turn sharply to the left if the angle error is large.)
- **If distance is 'close', then linear_velocity is 'slow'.** (Regardless of the angle, slow down when near the goal to avoid overshooting.)

These rules form a robust rule base that ensures the robot behaves predictably under various conditions.

2.3 Defuzzification

Defuzzification is the final step, where the fuzzy outputs are converted back into a single crisp value. The controller uses the **Centroid (Center of Gravity)** method to determine the final values for linear and angular velocities, which are then applied to the robot’s kinematic model.

3 Simulation and Results

The simulation was conducted in a two-phase approach to ensure the robot not only reached the target position but also achieved the correct final orientation.

Phase 1: Position Control The robot’s state was updated iteratively using a small time step ($\Delta t = 0.1s$). In each step, the controller calculated the distance and angle error, applied the fuzzy rules, and updated the robot’s position and orientation. This phase continued until the robot’s distance to the goal fell below a predefined threshold of 0.05 meters.

Phase 2: Orientation Control Once the robot was within the target position, it stopped forward motion ($v = 0$) and began to correct its final orientation. A simple proportional controller was used to adjust the angular velocity (ω) based on the remaining angle error until it was within an acceptable threshold of 0.05 radians (approximately 3 degrees).

The resulting trajectory shows the robot successfully navigating from the origin to the target position and then rotating to the final orientation. The path is smooth, demonstrating the effectiveness of the fuzzy controller in providing continuous and stable control signals.

4 Conclusion

The fuzzy logic controller successfully guided the unicycle robot to the specified goal state. The use of fuzzy logic allowed for the creation of an intuitive, rule-based system that handles the non-linear dynamics of the robot’s movement effectively. The two-phase simulation approach ensured that both the positional and orientational goals were met, resulting in a successful and well-defined trajectory.

5 Simulation Results

The following figure shows the trajectory of the robot from its initial position to the final goal state.

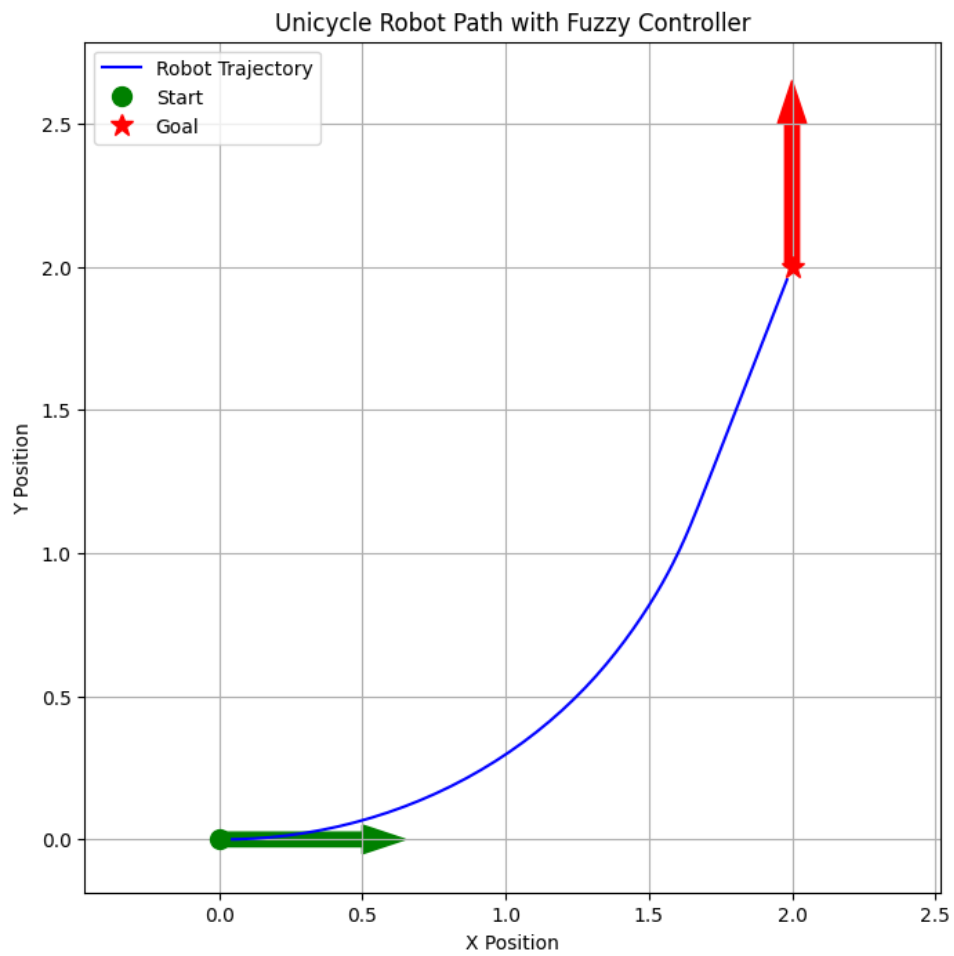


Figure 1: The robot's trajectory from $(0,0)$ to the goal state $(2, 2, \pi/2)$.

Project Link

- [Google Colab Notebook](#)